Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Organización de Lenguajes y Compiladores 2



Manual Técnico - St4ck Over Pig

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Octubre de 2026.

Índice

Descripción....3
1. Stack Tecnológico....3
2. Archivo de Configuración....3
3. Requisitos del Sistema y Entorno de Desarrollo....5
3.1 Dependencias....6
3.2 Plugins y generación....6
4. Compilación y Generación del Ejecutable....6
Opción A: Usando Maven....6
Opción B: Usando IntelliJ IDEA....7
5. Arquitectura General del Sistema....7
5.1 Presentación: com.compi.frontend....7
5.2 Fachada y selección de lenguaje....7
5.3 Análisis, semántica y generación....8
5.4 Errores y runtime de análisis....8
5.5 Flujo de datos....8
6. Gramáticas....8
6.1 PigLatin — PigLatin.g4....9
6.2 Expresiones y sentencias PigLatin:....9
6.3 Y — Y.g4....10
6.4 Zetariano — Zetariano.g4....11
7. Semántica y código intermedio....12
8. Back-end de ensamblador RISC-V....13
8.1 Clases....13
8.2 Modelo de datos....14
8.3 Syscalls (RARS)....14
8.4 Registros....15
8.5 Runtime inline....15
8.6 Ejecución interna (RiscvRunner)....15
9. Integración con el IDE....16
10. Diagrama de clases....16

Manual Técnico: St4ck Over Pig

Descripción

Stack Over Pig es un entorno de escritorio para editar y compilar programas en PigLatin, Y y Zetariano. Integra un editor con pestañas y resaltado, árbol de proyecto, generación de diagnósticos, tabla de símbolos, árbol de análisis, traza de parser y salidas intermedias. Los archivos se asocian a su lenguaje mediante extensión: .pig, .y y .z.

El procesamiento común implementado por Compiler coordina análisis léxico y sintáctico ANTLR, a semántica de Zetariano (encapsulamiento, herencia, polimorfismo con @Override y this), objetos de PigLatin con tipo explícito y null, y la traducción de las cuartetas/tripletas a ensamblador RISC-V funcional (módulo nuevo com.compi.backend.riscv, independiente de los emisores de C).

1. Stack Tecnológico

El proyecto está desarrollado enteramente bajo un entorno de arquitectura modular en Java, integrando herramientas estándar de la industria para el análisis léxico, sintáctico y el diseño de interfaces gráficas:

- **Lenguaje: Java 22 (JDK 22)** → Utilizado para aprovechar las características modernas del lenguaje, gestión de memoria y el rendimiento del motor de ejecución.
- **Analizador Léxico y Sintáctico: ANTLR 4 (ANother Tool for Language Recognition)** — Empleado para la definición de la gramática del lenguaje (.g4), generación automática del *Lexer*, *Parser* y el recorrido del Árbol de Análisis Sintáctico (AST) mediante el patrón *Visitor*.
- **Interfaz Gráfica de Usuario (GUI): Java Swing** → Framework nativo de Java para la construcción de la interfaz de escritorio, paneles de edición de código, consola de resultados y visualización de tablas de símbolos y errores.
- **Librería de Apariencia (Look and Feel): FlatLaf** → Integrada para dotar a la interfaz de Java Swing de un diseño moderno, limpio y profesional (compatible con temas oscuros y claros), elevando la experiencia visual del IDE.
- **RARS 1.6:** Ejecución del ensamblador dentro del IDE.

2. Archivo de Configuración

A continuación se presenta la estructura completa del archivo de configuración utilizada en el proyecto para asegurar la correcta compilación y resolución de dependencias:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.compi</groupId>
  <artifactId>contacto-3xtrat3rr3str3D</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>org.netbeans.external</groupId>
      <artifactId>AbsoluteLayout</artifactId>
      <version>RELEASE260</version>
    </dependency>
    <dependency>
      <groupId>com.formdev</groupId>
      <artifactId>flatlaf</artifactId>
      <version>3.6</version>
    </dependency>
    <dependency>
      <groupId>com.formdev</groupId>
      <artifactId>flatlaf-intellij-themes</artifactId>
      <version>3.6</version>
    </dependency>
    <dependency>
      <groupId>org.antlr</groupId>
      <artifactId>antlr4</artifactId>
      <version>4.13.2</version>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <version>1.18.34</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>

  <properties>
    <maven.compiler.source>22</maven.compiler.source>
    <maven.compiler.target>22</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <build>
    <plugins>
```

```
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.13.0</version>
          <configuration>
            <release>22</release>
          </configuration>
        </plugin>
        <plugin>
          <groupId>org.antlr</groupId>
          <artifactId>antlr4-maven-plugin</artifactId>
          <version>4.13.2</version>
          <executions>
            <execution>
              <id>antlr</id>
              <phase>generate-sources</phase>
              <goals>
                <goal>antlr4</goal>
              </goals>
            </execution>
          </executions>
          <configuration>
            <visitor>true</visitor>
            <listener>true</listener>
          </configuration>
        </plugin>
      </plugins>
    </build>
</project>
```

3. Requisitos del Sistema y Entorno de Desarrollo

Para compilar, modificar o ejecutar el código fuente de la aplicación, el equipo de desarrollo o despliegue debe contar con:

- **Java Development Kit (JDK 22):** Asegúrate de tener instalado JDK 22 y configurada la variable de entorno JAVA_HOME apuntando a su ruta de instalación. Puedes validarlo en la terminal con:

  java -version

- **Entorno de Desarrollo Recomendado:** IntelliJ IDEA (con soporte nativo para proyectos Java y plugins de ANTLR) o Apache Netbeans.

- **Gestor de Dependencias:** Maven o Gradle (según la configuración del proyecto para manejar las librerías de ANTLR runtime).

3.1 Dependencias

• org.netbeans.external:AbsoluteLayout:RELEASE260 — soporte de layouts de formularios.

• com.formdev:flatlaf:3.6 y flatlaf-intellij-themes:3.6 — apariencia Swing.

• org.antlr:antlr4:4.13.2 — ANTLR y runtime correspondiente.

• org.projectlombok:lombok:1.18.34 con scope provided — procesador/soporte de compilación, no dependencia de ejecución empaquetada.

3.2 Plugins y generación

maven-compiler-plugin 3.13.0 compila con release 22. antlr4-maven-plugin 4.13.2 se ejecuta en generate-sources mediante el goal antlr4; visitor y listener están habilitados. Así, las clases Lexer, Parser, BaseVisitor/Visitor y Listener se derivan de las gramáticas antes de compilar el código Java que las utiliza.

Las salidas de ANTLR son generadas: no edite a mano clases generadas dentro de target. Modifique el .g4 y vuelva a compilar. El directorio target puede eliminarse y reconstruirse con Maven.

mvn clean package

4. Compilación y Generación del Ejecutable

Para empaquetar la aplicación de escritorio junto con todas sus dependencias (incluyendo ANTLR) en un único archivo ejecutable:

Opción A: Usando Maven

cd contacto-3xtrat3rr3str3D mvn clean package
clean elimina la salida previa, generate-sources genera analizadores ANTLR y package compila las fuentes y empaqueta el artefacto Maven. El resultado se deposita en target/contacto-3xtrat3rr3str3D-1.0-SNAPSHOT.jar. El pom actual no configura Shade, Assembly ni un plugin de JAR ejecutable con dependencias; por eso ese JAR es un artefacto normal y no debe asumirse autocontenido. El pom tampoco configura pruebas o un plugin de empaquetado nativo .exe/.msi.
La clase de entrada de la aplicación es com.compi.Main. Una forma directa de ejecutarla desde el proyecto es usar IntelliJ IDEA, que resuelve dependencias Maven, o invocar Maven con un plugin de

ejecución si está disponible/configurado en el entorno. No se debe prometer java -jar para el JAR generado por el pom actual: no define Main-Class ni incorpora automáticamente dependencias.

Opción B: Usando IntelliJ IDEA

1. Ve a **File > Project Structure > Artifacts**.
2. Añade un nuevo artefacto de tipo **JAR > From modules with dependencies...**
3. Selecciona la clase principal (Main) que inicializa la interfaz de Java Swing.
4. Selecciona **Build > Build Artifacts...** y compila. El archivo estará disponible en la carpeta out/artifacts/.

Para ejecutar la aplicación empaquetada desde cualquier terminal compatible con Java 22, utiliza:

java -jar Contacto_3xtr4t3rr3str3D.jar

Para distribuir un ejecutable autocontenido, hace falta añadir y validar configuración de empaquetado (por ejemplo, un JAR con dependencias o jpackage). Esa configuración no está presente en el pom inspeccionado; documentar este límite evita presentar el JAR delgado como una aplicación lista para doble clic.

5. Arquitectura General del Sistema

El software está estructurado bajo una arquitectura limpia y modular dividida en tres capas principales: la definición formal de la gramática, la interfaz gráfica de usuario (**Frontend**) y el motor de procesamiento, análisis e interpretación (**Backend**).

5.1 Presentación: com.compi.frontend

- MainWindow compone la ventana principal, cablea las acciones de usuario y solicita compilaciones. TopToolbar contiene acciones y controles. FileTreePanel gestiona la raíz, el filtro y el árbol; EditorTabs administra EditorPanel por pestaña; ConsoleDock presenta las seis áreas de salida; ToolWindow y ToolWindowDock agrupan AST, símbolos, pila y errores; StatusBar refleja posición, lenguaje, resultado y contadores. SyntaxHighlighter y LineNumberComponent apoyan la edición.

5.2 Fachada y selección de lenguaje

- Compiler es la fachada central usada por la ventana para compilar y obtener resultados. LanguageCompilerFactory asocia identificadores/extensiones con PigLatinCompiler, YCompiler y ZetarianoCompiler, que implementan LanguageCompiler a través de

AbstractLanguageCompiler. Esta frontera encapsula parseo, vocabulario/reglas, imports y visitas específicas del lenguaje. CompilerFacade es otra fachada presente en el repositorio con métodos explícitos por lenguaje; el flujo de la interfaz actual se apoya en Compiler.

5.3 Análisis, semántica y generación

- ANTLR produce el parse tree; cada *AstBuilder construye el **AST propio** (com.compi.backend.ast). La semántica (AstSemanticAnalyzer, AstExpressionAnalyzer, AstDefinitionAnalyzer, AstDeclarationAnalyzer y TypeCompatibility) alimenta SymbolTable/TypeTable y produce CompilationError. AstCodeGenerator emite operaciones a C3DGenerator/IntermediateCodeManager; las cuartetas sirven de base para tripletas, **C** (CCodeEmitter → FlatEmitter) y **RISC-V** (RiscvAssembler). ParseTrace produce pasos para el panel Pila y ASTDotGenerator el DOT de la vista AST.

5.4 Errores y runtime de análisis

- AntlrErrorCollector adapta errores de lexer/parser a CompilationError con categoría y posición. Los resultados se acumulan para exponer diagnósticos y métricas. Stack, StackSimulator y StackState representan/simulan la pila de procesos mostrada por el frontend. El nombre runtime en este subsistema corresponde a la simulación/visualización que realiza el compilador, no a una máquina virtual general de ejecución de programas.

5.5 Flujo de datos

- Archivo fuente o texto del editor → LanguageCompilerFactory selecciona compilador.
- Lexer ANTLR → tokens; parser ANTLR → árbol sintáctico y errores léxicos/sintácticos.
- Carga de imports cuando corresponda → análisis semántico y tabla de símbolos.
- Visitor del lenguaje → instrucciones de tres direcciones/cuádruplas → tripletes.
- Resumen del flujo: fuente → frontend → backend/ANTLR → análisis → generación → vistas.

6. Gramáticas

La gramática formal de **St4ck Over Pig** está estructurada mediante tres lenguajes, PigLatin, Y, Zetariano, cada uno con sus reglas sintácticas (No Terminales) definidas en formato ANTLR4 separadas en archivos .g4 distintas. A continuación, se detalla el funcionamiento y propósito específico de cada producción que compone el analizador sintáctico:

6.1 PigLatin — PigLatin.g4

Entrada y unidades:

- init: raíz de entrada; requiere pig_latin.
- pig_latin: cero o más imports, sección opcional de variables y bloque principal maior.
- import_declaracion: palabra import seguida de una ruta formada por componentes VARIABLE separados por puntos; punto y coma opcional.
- variables: sección VARIABILES > seguida de una o más declaraciones o declaraciones de arreglos.
- declaracion: admite formas iniciadas por esto: valor tipado por tipo y expresión; texto/carácter explícito; construcción novus Tipo(argumentos); instancia de estructura; expresión; o arreglo.
- arreglo_declaracion: arreglo series con uno o más tamaños entre corchetes, tipo primitivo o tipo de estructura y literal opcional entre llaves.
- elemento_arreglo: lista separada por comas de valores que pueden ser expresiones o arreglos anidados.
- elemento_arreglo_valor: una expresión o una lista anidada recursiva.
- elemento_arreglo_struct: lista de instancias de estructura separadas por comas.
- structura_instanciacion: campos entre llaves con asignaciones de atributos; también permite la forma de referencia VARIABLE[indice]. El nombre de estructura puede ser opcional en la forma con llaves.
- atributo_asignacion: nombre de atributo y valor, que puede ser expresión, estructura, literal de arreglo o construcción novus.
- arreglo_literal: lista opcional de elementos de arreglo entre llaves.
- tipo_dato: NUMERUS, TEXTUM, DECIMALIS, LITTERA, VERUM o FALSUS; son los tokens de tipo que la gramática permite en ese contexto.

6.2 Expresiones y sentencias PigLatin:

- expresion: uno o más términos conectados por operadores aritméticos; la regla no expresa precedencia por niveles separados.
- termino: variable, acceso, literal numérico/texto/carácter/booleano, llamada, construcción novus o término con signo menos.
- acceso_miembro: variable seguida de uno o más accesos por punto o índice entre corchetes.
- arreglo_acceso: variable indexada con expresión y posible acceso de miembro adicional.
- operacion_aritmetica: suma, resta, multiplicación o división.
- maior: palabra MAIOR y >, cero o más sentencias, FINIS y punto y coma; delimita el cuerpo principal.
- sentencia: impresión, lectura, asignación, condicional, ciclos dum/facere/per, salto, llamada o incremento/decremento abreviado.
- leer_sentencia: operador << con destino opcional de variable o miembro.
- asignacion_sentencia: destino variable/miembro asignado a expresión, condición, estructura, arreglo o llamada novus; punto y coma opcional.
- imprimir_sentencia: operador >> aplicado a uno o más valores/cadenas/llamadas, separando elementos con nuevas apariciones del operador; punto y coma opcional.
- si_sentencia: condición entre paréntesis, bloque principal, cero o más ramas aliter con condición y rama ALITER final opcional; bloques entre llaves y cierre FINIS;.
- aliter_bloque: rama condicional alternativa ALITER(condición) con bloque.
- ciclo_dum: ciclo con condición antes del bloque (while), cerrado con FINIS;.

- ciclo_facere: bloque ejecutado antes de la condición DUM (do-while), con punto y coma final.
- ciclo_per: bucle con inicialización, condición e incremento, dentro de paréntesis, más cuerpo entre llaves.
- inicializacion_per: declaración tipada o asignación inicial.
- condiciones_per: una condición lógica.
- incremento_per: incremento/decremento abreviado o asignación de expresión.
- salto_sentencia: PERGE; o INTERRUMPE;, correspondientes a continuar/interrumpir según la semántica del visitor.
- condicion: conjunciones unidas por OR; la recursión organiza disyunción.
- conjuncion: negaciones unidas por AND.
- negacion_logica: operador non aplicado recursivamente o primaria lógica.
- primaria_logica: condición agrupada, comparación entre expresiones, booleano, variable o llamada.
- operador_relacional: >=, <=, ==, !=, < o >.
- llamada_funcion: función o miembro invocado con argumentos opcionales.
- argumentos: una o más expresiones separadas por coma.
- Léxico PigLatin: define palabras reservadas latinas y operadores simbólicos; VARIABLE sigue identificadores ASCII con guion bajo opcional, enteros y decimales usan dígitos, cadenas usan comillas dobles y carácter comillas simples. WS y comentarios de línea se omiten; comentarios ##...## van a canal oculto.

6.3 Y — Y.g4

- La gramática declara tokens INDENT y DEDENT y las reglas de parser dependen de NUEVA_LINEA más esos tokens. YIndentLexer implementa el tratamiento de sangría requerido; no se debe sustituir por el lexer generado sin esa lógica en el flujo de compilación.
- init: raíz, exige archivo y EOF para consumir toda la entrada.
- archivo: sección de estructuras opcional antes de funciones, o solamente sección de funciones.
- estructuras_seccion: marcador %estructuras, saltos opcionales y una o más definiciones separadas por saltos.
- estructura_def: estructura con campos indentados; variante sin bloque indentado; o forma de estructura anidada/variable.
- estructura_anidada_o_variable: dos identificadores, útil para tipo/nombre o declaración anidada en los contextos donde se acepta.
- campo_struct: campo de tipo primitivo o tipo identificado, con dimensiones de arreglo opcionales y terminación por saltos; admite forma anidada.
- funciones_seccion: marcador %funciones y una o más funciones.
- funcion_def: definir nombre(parámetros), retorno opcional -> tipo, dos puntos y bloque.
- parametros: lista de parámetros separada por comas.
- parametro: parámetro escalar o parámetro arreglo indicado por []/{} antes del tipo y nombre.
- bloque: una o más sentencias indentadas o una sentencia en línea.
- sentencia: declaración, asignación, if, elegir, para, mientras, hacer-mientras, retorno, romper, continuar, imprimir, llamada, incremento/decremento, estructura local o salto de línea; muchos puntos y coma son opcionales.
- declaracion_estructura_local: definición de estructura indentada dentro de un bloque.
- declaracion_variable: tipo básico o identificado, dimensiones opcionales e inicializador; también contempla formas de bool con/sin asignación.
- expresion_inicializacion: literal/lista anidada entre llaves o expresión normal.
- elemento_lista: expresión o lista anidada entre llaves.
- asignacion: asigna expresión a acceso de miembro, identificador o elemento indexado.
- acceso_miembro: navegación mediante puntos y/o índices; permite solo índices como destino.

- si_sentencia: if(condición), entonces/dos puntos opcionales y bloque; puede continuar con sino-si, sino condicionado o contrario.
- sino_si_bloque: rama SINO(condición) con bloque.
- sino_bloque: SINO condicionado o SINO simple, con bloque.
- contrario_bloque: palabra CONTRARIO opcionalmente seguida por dos puntos y bloque.
- saltos: cero o más tokens de nueva línea; separa construcciones en la gramática.
- elegir_sentencia: selección de expresión con casos y bloque SIEMPRE opcional; admite delimitadores de llaves y formato indentado.
- caso_bloque: CASO expresión: sentencias opcionales y ROMPER opcional.
- siempre_bloque: rama por defecto SIEMPRE con sentencias opcionales y ROMPER opcional.
- bloque_interno_opcional: cuerpo indentado no vacío o secuencia de sentencias en línea, posiblemente vacía.
- para_sentencia: bucle con inicialización/declaración, condición e incremento separados por punto y coma y bloque.
- mientras_sentencia: mientras(condición), opción HACER y bloque.
- hacer_mientras_sentencia: HACER bloque, luego MIENTRAS(condición) y punto y coma opcional.
- retornar_sentencia: RETORNAR con expresión opcional.
- romper_sentencia / continuar_sentencia: tokens de control de flujo correspondientes.
- imprimir_sentencia: imprimir(argumentos opcionales). llamada_funcion: ID(argumentos opcionales). argumentos: expresiones separadas por comas.
- incremento_decremento: ++/-- antes o después de identificador o acceso.
- expresion: expresión aritmética, relacional, lógica, negada, llamada o lectura. Las alternativas recursivas reconocen operadores, aunque la precedencia efectiva debe interpretarse junto al parser/visitor.
- termino: identificador/acceso, literal, booleano, agrupación, lista, signo unario o expresión de lectura.
- leer_funcion: leer() sin argumentos.
- condicion: expresión usada en contexto booleano.
- tipo_dato: entero, cadena, flotante, carácter, bool o identificador como tipo de usuario.
- operador_aritmetico, operador_relacional, operador_logico, operador_negacion: agrupan respectivamente + - * /; == != < >; && ||; !.
- Léxico Y: marcadores %estructuras y %funciones, palabras reservadas en español, identificadores, literales y operadores. NUEVA_LINEA captura salto seguido por espacios/tabuladores para cooperar con YIndentLexer. WS horizontal se omite y los comentarios // se omiten; /*...*/ se envían al canal oculto.

6.4 Zetariano — Zetariano.g4

- La sintaxis está inspirada en clases estilo Java, con palabras reservadas en inglés y delimitadores de llaves.
- programa: una o más definiciones de clase.
- claseDef: public opcional, class, identificador y cuerpo de miembros entre llaves.
- miembroClase: atributo, constructor o método.
- atributoDef: tipo, nombre, dimensiones de arreglo opcionales, inicializador opcional y punto y coma; incluye variante con corchetes antes del identificador.
- constructorDef: public opcional, nombre de constructor, parámetros opcionales y bloque.
- metodoDef: public opcional, tipo de retorno, nombre, parámetros opcionales y bloque.
- parametros: lista separada por comas; parametro: tipo y nombre.

- bloque: sentencias entre llaves o sentencia única.
- sentencia: declaración, asignación, if, switch, for, while, do-while, retorno, break, continue, llamada, incremento/decremento o sentencia vacía.
- declaracionVariable: tipo, nombre, dimensiones opcionales, inicializador opcional y punto y coma; admite ambas ubicaciones de corchetes.
- asignacion: destino simple/miembro con =, +=, -= o *=, o destino indexado con =.
- incrementoDecremento: variable/miembro con ++ o --.
- sentenciaIf: if(condición) bloque, cero o más else if y else opcional.
- sentenciaSwitch: switch(expresión), casos y default opcional dentro de llaves.
- casoSwitch: case expresión:, sentencias y break opcional.
- defectoSwitch: default:, sentencias y break opcional.
- sentenciaFor: for(init; condición; update) con componentes opcionales según reglas y bloque.
- forInit: declaración o asignación sin el punto y coma final incluido por su propia regla.
- asignacionExpresion: asignación simple o compuesta usada en cabecera de for.
- forUpdate: incremento/decremento o asignación.
- sentenciaWhile: while(expresión) bloque.
- sentenciaDoWhile: do bloque while(expresión);.
- retorno: return con expresión opcional y punto y coma.
- romper / continuar: break; y continue; respectivamente.
- llamadaFuncionSemilla: println, print, readln sin argumentos, llamada por ID o llamada a miembro; argumentos opcionales donde aplica.
- argumentos: una o más expresiones separadas por coma.
- expresion: alternativas etiquetadas para ternario, multiplicación/división/módulo, suma/resta, relacionales, igualdad, AND/OR, negación, menos unario, prefijo ++/--, creación de objeto/arreglo, literal de arreglo, null, acceso, literales, booleanos, agrupación y llamadas.
- literales de arreglo: uno o más elementos expresión/arreglo anidados entre llaves y separados por coma.
- accesoMiembro: identificador seguido de uno o más accesos miembro.
- miembroAcceso: punto más identificador con índices opcionales, o uno o más índices.
- tipoDato: int, double, char, boolean, String, void o identificador de tipo.
- Léxico Zetariano: palabras reservadas inglesas, identificadores, literales numéricos/textuales, operadores y puntuación. WS y comentarios // se omiten; comentarios /*...*/ se envían al canal oculto. ZetarianoCompiler valida además la correspondencia del nombre del archivo con el de la clase declarada, requisito semántico/operativo adicional al patrón de parser.

7. Semántica y código intermedio

- La semántica es compartida por los tres lenguajes (mismo AST):
- Encapsulamiento (Zetariano): `private` solo desde la clase que declara; `protected` desde esa clase o subclases; `public` sin restricciones; `default` (sin modificador) es *package-private*. Zetariano no declara paquetes, por lo que todas las clases de una compilación pertenecen al mismo paquete y un miembro `default` puede usarse desde cualquier clase del programa. No se interpreta como `private`. El enunciado lo define así: default (sin modificador): Accesible únicamente dentro de las clases que pertenecen al mismo paquete (también llamado package-private).
- Herencia: `parentClass` y cadena de superclases; `getMember` desciende por ancestros; `@Override` obligatorio para sobrescribir; campos del ancestro primero y la subclase después (`inheritedFieldCount`).

- Despacho virtual: cada clase recibe una marca de tipo única guardada en otype[base] (TYPE_SET/TYPE_GET); al llamar un método polimórfico se bifurca por marca a la etiqueta concreta. La llamada estática se conserva cuando todas las clases resuelven el mismo método.
- this: variable this del tipo de la clase; identificadores no resueltos caen a campos heredados sin calificar.
- PigLatin: objetos con tipo explícito (novus C(...) : Tipo), objetos null : Tipo, y compatibilidad subclase→superclase en asignación.
- Generación C3D: el AST → cuartetas (C3DGenerator) con el modelo plano: stack/heap/otype en celdas double, escalares P/H/RET/RA, frames por invocación y despachador por retorno.

8. Back-end de ensamblador RISC-V

Módulo nuevo (com.compi.backend.riscv, 14 clases) que consume las mismas cuartetas que FlatEmitter y produce un .s funcional para RARS (RV32IMF/D). No reutiliza los emisores de C. Además de generarlo, el módulo lo ensambla y ejecuta internamente (sección 9.6) sin salir del IDE.

8.1 Clases

Clase	Responsabilidad
RiscvAssembler	Fachada generateAssembly(cuartetas,tabla)→ texto .s
RiscvTranslator	Orquesta: main, cuerpo, funciones y secciones
RiscvLayout	Clasifica la IR: cuerpos, slots, frames, globales
RiscvConstants	Piscina de constantes .double inicializadas en .data
RiscvContext	Estado por compilación (escritor, layout, pila de expresión)
RiscvAccess	Helpers: variables, frames, constantes, heap, concat
RiscvFlow	Saltos, CALL/RETURN, despacho por RA, fin de programa
RiscvArithmetic	Asignaciones, operaciones y concatenación
RiscvMemory	HEAP_GET/SET y ARRAY_GET/SET (completitud)
RiscvRuntime	Rutinas inline: imprimir, leer, number_to_text
RiscvWriter	Acumula las líneas del ensamblador
RiscvRunner	Ensambla y ejecuta el .s con RARS embebido; admite parada cooperativa por tramos

InteractiveInput	InputStream bloqueante para la entrada interactiva (una línea por lectura; close() desbloquea)
StreamingOutputStream	ByteArrayOutputStream que publica cada fragmento de salida en cuanto se produce

Nota de compatibilidad RARS: las constantes se emiten dentro de .data porque RARS no reconoce .rodata, y las ramas condicionales del lenguaje se emiten como «condición invertida hacia una etiqueta local + j incondicional» porque RARS marca como error fatal las B_FORMAT con objetivo a más de ±4096 bytes y no las relaja. Así, cualquier .s generado (aun con funciones o ciclos muy grandes) ensambla en el simulador interno.

8.2 Modelo de datos

stack/heap/otype (800 000 celdas double cada uno), escalares H/P/RET/RA, frames por invocación, argumentos en P+2+i, marca de retorno en P+0 y P previo en P+1; despacho L_dispatch por (int)RA; cadenas en heap terminadas en 0; null = -1.0.

8.3 Syscalls (RARS)

Servicio	**a7**	**Argumentos**
Imprimir entero	1	a0
Imprimir doble	3	fa0
Imprimir cadena	4	a0
Leer entero	5	a0
Leer doble	7	fa0
Leer cadena	8	a0, a1
Imprimir carácter	11	a0
Leer carácter	12	a0
Salir	10	—

8.4 Registros

Scratch t0..t2/ft0..ft1 en los helpers; estado de rutinas en s1..s6, ft2..ft7, fa1/fa2; fa0 como registro de valor; a0/a7 para ecall; ra libre (rutinas inline, sin jal).

8.5 Runtime inline

print_string (carácter a carácter hasta 0/-1), print_char, read_text (buffer temporal + recorte de \n), read_char, number_to_text (dígitos enteros, . y hasta 12 fracciones). La parte entera se obtiene con fcvt.w.d ..., rtz para truncar sin perder la parte fraccionaria; así se imprimen valores como 77.5 y 1627.5. Todo inline para dejar ra libre.

8.6 Ejecución interna (RiscvRunner)

com.compi.backend.riscv.RiscvRunner es el puente hacia el simulador RARS embebido (lib/rars1_6.jar, paquete público rars.api):

1. Crea un rars.api.Program nuevo por ejecución con Options (startAtMain = true — arranca en main: —, pseudo = true y maxSteps = 500 000 por tramo; el total se acota a 20 000 000).
2. assembleString(ensamblador) ensambla desde el texto; los errores se convierten en un diagnóstico conciso (Línea N: mensaje) en Result.
3. setup(args, stdin) inyecta la entrada del usuario y simulate() corre por tramos hasta terminar.
4. Result(ok, stdout, stderr, reason, exitCode, elapsedMillis, message) captura la salida y la razón de terminación (NORMAL_TERMINATION, MAX_STEPS, DETENIDO...).

Parada cooperativa (run(assembly, input, stopRequested)): en vez de una sola llamada a simulate(), se ejecuta en tramos de CHUNK_STEPS (500 000 instrucciones) y se consulta un BooleanSupplier entre tramos. Es la vía segura con la API pública: Simulator.stopExecution() existe, pero Program relee su hilo interno tras simulate() y esa llamada desde el EDT puede provocar una condición de carrera (NullPointerException). El Program se crea una vez y las llamadas sucesivas a simulate() continúan desde el PC actual, acumulando la salida; cuando la solicitud es verdadera se devuelve reason = "DETENIDO" con la salida parcial. CompilationActions mantiene la bandera volatile stopRequested (escrita en el EDT, leída en el hilo de trabajo, sin tocar estructuras de RARS desde varios hilos).
Entrada y salida interactivas (runInteractive(assembly, input, output, stopRequested)): en vez de una entrada fija, la lectura se sirve bajo demanda. RARS atiende los ecall de lectura por SystemIO.getInputReader() y los de impresión por SystemIO.getOutputWriter(); ambos son perezosos y, sin GUI de RARS, se crean sobre System.in/System.out. Por eso la variante interactiva llama a setup(args, null) (paso directo) y sustituye temporalmente System.in por un InteractiveInput y System.out/System.err por StreamingOutputStream alrededor del bucle de simulate(), restaurándolos en finally:

- InteractiveInput es un InputStream respaldado por una cola: el hilo que simula se bloquea hasta que la interfaz submit una línea; read(byte[],…) devuelve solo el resto de la línea ya disponible para que BufferedReader vea el fin de línea sin esperar otra. close() marca EOF y desbloquea la lectura (lo invoca Detener), de modo que también se puede detener un programa que está esperando datos.

- StreamingOutputStream publica cada fragmento escrito (vía Consumer<String>) además de acumularlo: así el usuario ve los mensajes del programa —y en particular la petición de datos— antes de responder.

No se usa reflexión: es el propio diseño de RARS en modo sin GUI el que toma System.in/System.out de forma perezosa. La sustitución de los flujos estándar es segura porque las ejecuciones ya están serializadas (un Program en setup a la vez, sobre un SwingWorker).
Solo puede haber un Program en setup a la vez; por eso cada ejecución crea uno nuevo y CompilationActions impide lanzamientos simultáneos (corre sobre un SwingWorker para no bloquear la interfaz). RARS se distribuye con licencia MIT y no está en Maven Central, de ahí el systemPath al jar versionado.

9. Integración con el IDE

- Compiler: genera el ensamblador en publishIntermediate (junto al C), getRiscvCode(), setRiscvEnabled(boolean)/isRiscvEnabled() y limpieza en reset. La generación es independiente del modo Tripletes/Cuartetas.
- ConsoleDock: vistas Ensamblador (View.ASSEMBLY) y Simulador (View.RUN, con salida de ejecución y campo de entrada) entre "Código C" y "Resumen"; setRunOutput/appendRunOutput/getRunInput para el ciclo de ejecución y setOnRunInputSubmit/setRunInteractive para la entrada interactiva (Enter envía la línea; Mayús+Enter inserta salto; botón Enviar).
- CompilationActions: publica riscvCode en la vista y escribe NombreDelArchivo.s en la raíz del proyecto junto al .c; runOrStopRiscv() (botón Ejecutar/Detener y Ctrl+R) arranca runRiscv() o, si ya hay una ejecución, marca stopRequested y cierra el InteractiveInput para desbloquear una lectura pendiente; runRiscv() lanza runInteractive en un SwingWorker, selecciona la vista Simulador y publica por fragmentos la salida (streamRunOutput), además de la razón de terminación y el estado en la barra inferior.
- TopToolbar/MainWindow: casilla RISC-V y botón Ejecutar en la barra superior (activados por defecto, con atajo Ctrl+R), cableados desde el constructor de MainWindow (sin tocar initComponents). setRunningState alterna el mismo botón entre Ejecutar y Detener según haya o no una simulación en curso; IdeIcons.stop() dibuja el cuadro.

10. Diagrama de clases

Adjunto a este documento podrá encontrar la imagen en formato svg y png, donde se puede visualizar de mejor manera cada clase y sus relaciones.